

Chapter 2: Types of Attacks & Malware



Agenda

1. Threats, Weaknesses, Vulnerabilities & Attacks

- Concepts: threats, weaknesses, vulnerabilities, attacks
- Common threat types
- Attack classifications

2. Attack Support Tools

- Vulnerability & weakness scanning tools
- Port scanning tools
- Sniffing tools
- Keyloggers

3. Common Attack Types

- Password attacks
- Malware attacks
- Denial of Service (DoS)
- Distributed DoS (DDoS)
- IP spoofing
- Eavesdropping
- Man-in-the-Middle (MITM)
- Email bombing & spam
- Other attack types

5. Malware Types

- Classification
- Malware descriptions
- Malware prevention & defense

5. Review Questions

- End-of-chapter review and self-assessment questions

Chapter 2: Types of Attacks and Malware

Exploring the **modern threat landscape** — from vulnerabilities to attack methodologies and malware defense.



Core Concepts

Threats, vulnerabilities,
attack fundamentals



Attack Tools

Scanning, sniffing,
reconnaissance



Attack Families

DoS, MITM, injection,
social engineering



Malware Analysis

Types, behaviors,
countermeasures

Threat: What Can Cause Harm?

A **threat** is any **potential** cause of an **unwanted incident** that may result in harm to a system or asset.

System weakness

System weakness category	Root cause	Concrete example (specific scenario)
Design flaw	Security controls missing in the architecture/logic from the start	An “internal-only” admin API is designed without authentication because it is assumed to be used only inside the LAN, but later it is accidentally exposed to the Internet
Installation / deployment mistake	Insecure defaults, missed hardening steps during setup	A database is installed and left with a default admin account / weak default password , or a sample admin console remains enabled
Programming / coding defect	Missing input validation, unsafe functions/libraries, logic bugs	A web form does not validate or sanitize input , enabling SQL Injection or other injection-style bugs
Administration / configuration error	Misconfiguration, excessive privileges, overly permissive access rules	A firewall rule allows 0.0.0.0/0 access to a management port (SSH/RDP/admin panel) that should be restricted to a trusted subnet
Hardware module weakness	Weakness in hardware/firmware design or update mechanism	A device firmware update mechanism does not verify digital signatures , so a malicious firmware image can be loaded (illustrative case)
Software module weakness	Bugs, insecure dependencies, unpatched components	An application uses an outdated dependency with a known vulnerability and patching is delayed , leaving a known exploit path open

Weakness vs Vulnerability



Weakness

Any **flaw, defect, or shortcoming** in design, implementation, or configuration

- Default admin passwords
- Missing input validation
- Plaintext data storage
- Excessive user privileges



Vulnerability

A weakness that is **actively exploitable** by a threat actor

- Known credentials exposed remotely
- Unpatched software flaw
- Misconfigured access control
- Context + exploitability = vulnerability



Key Takeaway: Every **vulnerability** is a weakness, but not every weakness is exploitable.

Weakness vs Vulnerability

Criteria	System Weakness (Weakness)	Security Vulnerability (Vulnerability)	Concrete example
Definition	A flaw/defect existing in a system (design, implementation, configuration, administration, hardware/software).	A weakness that can be exploited by an attacker to harm confidentiality, integrity, or availability (CIA) .	Weakness: an input field accepts any characters. Vulnerability: attacker exploits it as SQL Injection to read/delete data.
Exploitability	May or may not be exploitable (some weaknesses are harmless or hard to reach).	By definition, it is exploitable (or realistically exploitable under certain conditions).	Weakness: debug page exists but is not reachable externally. Vulnerability: debug page exposed publicly → attacker uses it to access system info.
Focus	Describes what is wrong in the system (a defect or misfeature).	Describes the security impact of that weakness when exploited (CIA damage).	Weakness: weak password policy. Vulnerability: credential guessing becomes feasible → account compromise (confidentiality/integrity loss).

Vulnerability Severity Levels

Depending on **exploitability**, security vulnerabilities have different **severity levels**.

Microsoft Classification

Critical – Dangerous

Can be exploited remotely, requires no authentication, causes immediate severe damage.

Moderate – Medium

Impact is limited by factors such as authentication requirements or complex exploitation conditions.

Important – Significant

Exploitation can affect the confidentiality, integrity, or availability of data.

Low – Low

Very difficult to exploit, limited impact, minimal effect on the system.

Other Organizations' Classification

High – Critical

Serious vulnerability that needs to be patched immediately to prevent potential attacks.

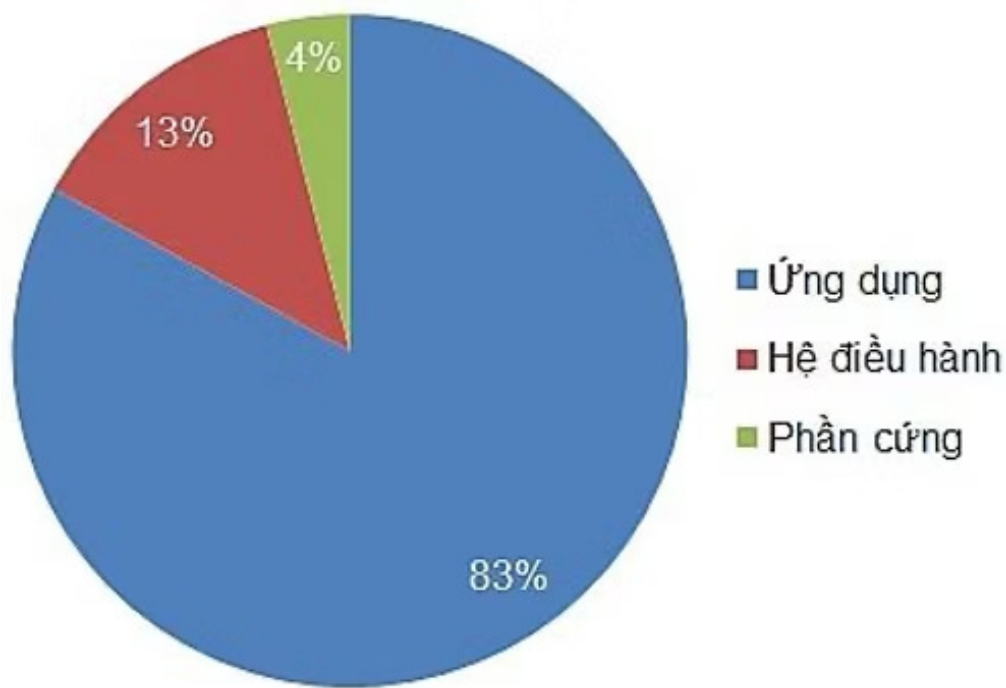
Medium – Moderate

Concerning vulnerability that should be addressed soon within a regular patch schedule.

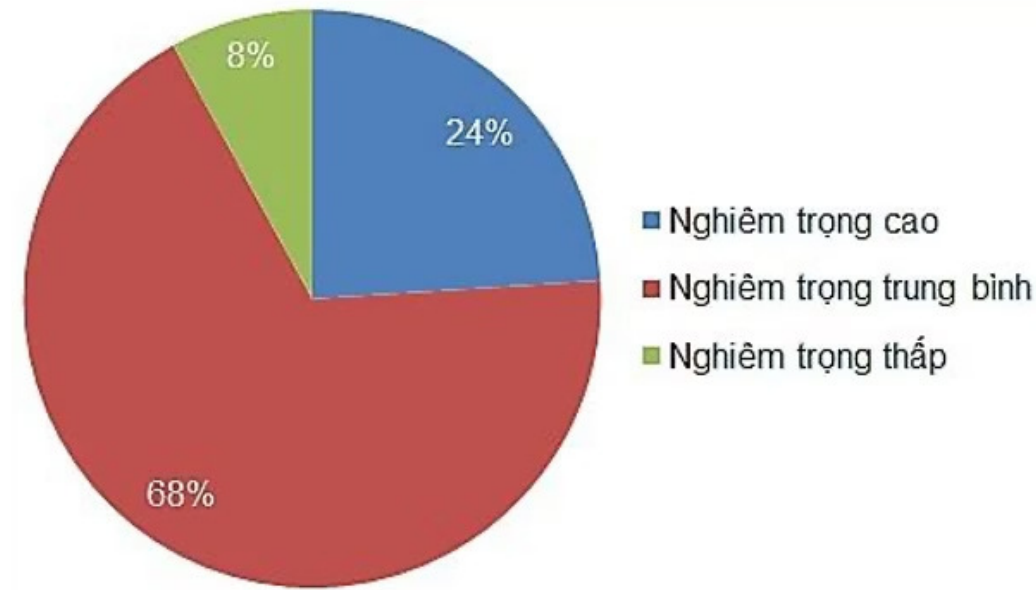
Low – Low

Low-risk vulnerability that can be handled according to a routine maintenance schedule.

2014 Statistics



Phân bố lỗ hổng bảo mật trong các thành phần của hệ thống



Phân bố lỗ hổng bảo mật theo mức độ nghiêm trọng

What Is an Attack?

ATTACK = THREAT + VULNERABILITY

Remove either component → attack cannot succeed



Threat Actor

- **Motivation** to cause harm
- **Capability** to exploit



Vulnerability

- **Exploitable** weakness
- **Entry point** for attack



Attack

- **Deliberate** action
- **Policy violation**

CIA Triad Impact



Confidentiality

- **Credential theft**
- **Data exfiltration**
- **Eavesdropping**



Integrity

- **Database tampering**
- **Malware infection**
- **Transaction manipulation**



Availability

- **Denial-of-service**
- **Ransomware encryption**
- **Infrastructure destruction**



Key Takeaway: Attacks are **deliberate actions**; **vulnerabilities** enable them.

Attack Objectives: Four Core Categories

Each attack objective maps directly to a violation of the **CIA Triad**.



Fabrication

- Creates **false/counterfeit** objects
- Violates **Integrity**
- *e.g., email spoofing, forged credentials*



Interception

- **Unauthorized access** to assets
- Violates **Confidentiality**
- *e.g., network sniffing, eavesdropping*



Interruption

- Disrupts **service availability**
- Violates **Availability**
- *e.g., DDoS, ransomware encryption*



Modification

- **Unauthorized alteration** of data
- Violates **Integrity**
- *e.g., SQL injection, record tampering*

 **Key Takeaway:** Attack objectives map directly to **CIA violations** — understanding them enables precise threat modeling and control design.

Passive vs. Active Attacks

Passive Attacks

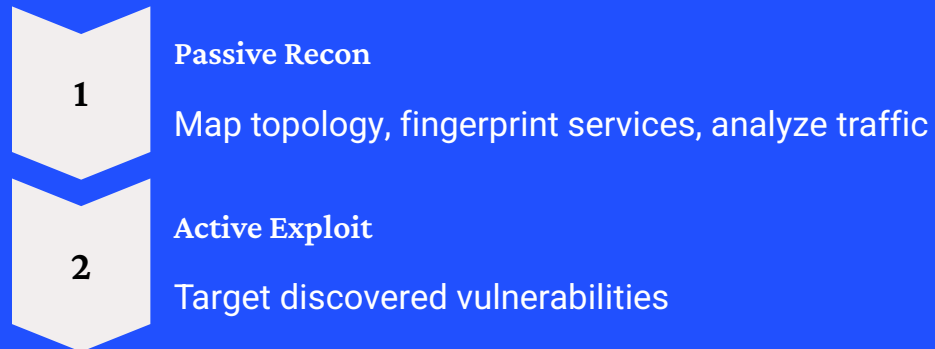
- **No system modification**
- Hard to detect, minimal traces
- Goal: **intelligence gathering**
- Traffic analysis, eavesdropping, port scanning
- Detection: **anomaly detection**, encryption verification

Active Attacks

- **Modifies systems, data, or services**
- Easier to detect via logs & monitoring
- Goal: **compromise, disrupt, or modify**
- MITM, DoS, SQL injection, malware
- Detection: **IDS/IPS**, integrity checks, access logs



Attack Progression Pattern



Defense Implications

- **Passive:** behavioral analysis, anomaly detection
- **Active:** IDS/IPS, integrity monitoring, access controls
- Signature-based tools **miss passive recon**
- Layered defenses address both stages

 **Key Takeaway:** Passive attacks gather intel; active attacks change or break systems.

Attacking assistant tools

Attacking assistant tools

The same tools serve both attackers and defenders — **context, authorization, and intent** determine ethical use



Vulnerability & Port Scanning

Identifying weaknesses and open services

Tools: Nessus, Nmap, OpenVAS



Sniffing & Packet Capture

Intercepting network traffic

Tools: Wireshark, Tcpdump, Pcap



Keyloggers

Capturing user input and credentials

Types: Hardware & Software keyloggers



Password Attacks

Breaking authentication mechanisms

Methods: Dictionary attacks, Brute force

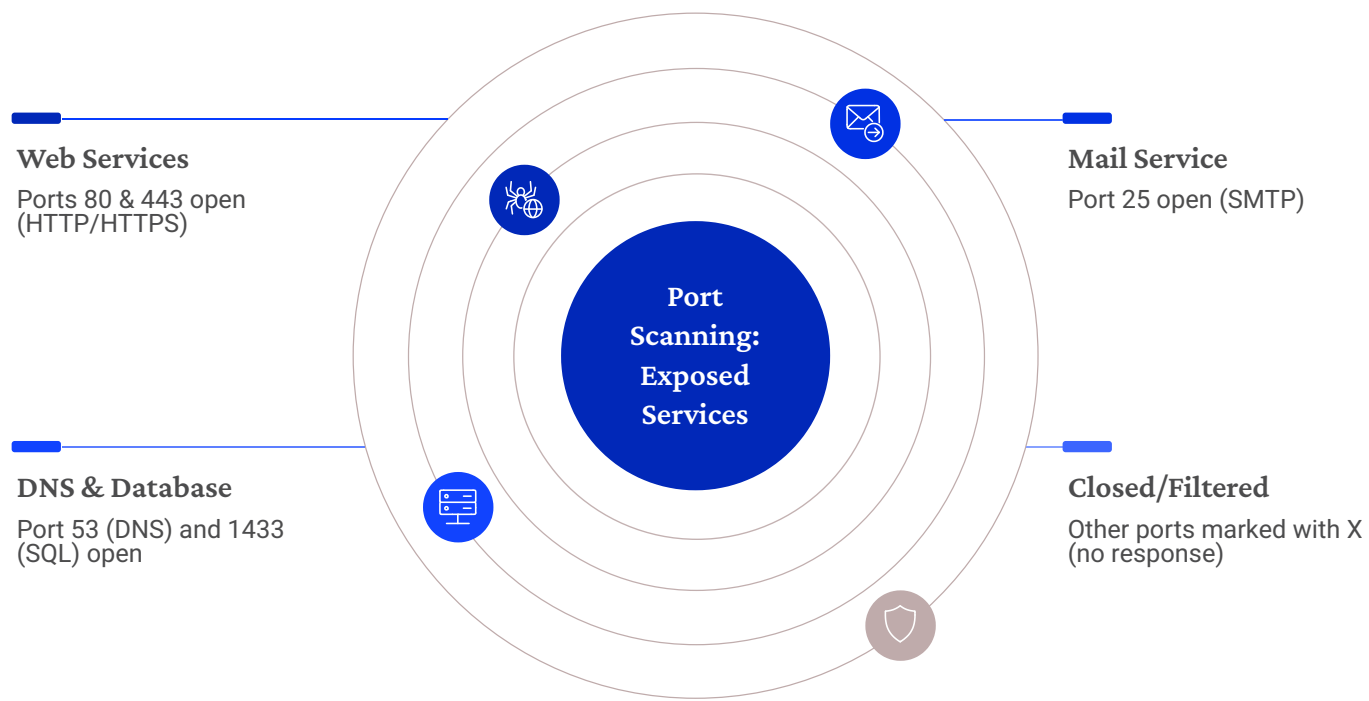
1. Vulnerability Scanning: Systems & Web Applications

Automated tools that identify **known weaknesses**, **misconfigurations**, and **missing patches** — enabling organizations to find and fix vulnerabilities before attackers exploit them.

Tool category	Primary purpose	Scope	Typical findings	Output / deliverables	Example
System vulnerability scanning tools	Proactively identify system weaknesses and security vulnerabilities in hosts	Operating systems and host configurations (typically Windows/Linux/multi-OS), system services	Missing patches, insecure configurations, exposed services, known system-level vulnerabilities	Detailed analysis per issue + remediation guidance (how to fix/patch/harden)	Microsoft Baseline Security Analyzer (MBSA) (Windows), Nessus Vulnerability Scanner (multi-OS)
Web application vulnerability scanning tools	Scan and analyze websites/web apps to find application-layer security flaws	Web applications, web pages, web endpoints and inputs	XSS, SQL Injection, CSRF, session/security misconfigurations	Vulnerability report + severity/technical details; often includes guidance to fix application flaws	Acunetix Web Vulnerability Scanner , IBM AppScan , Beyond Security AVDS , SQLmap

2. Port Scanning

Systematic probing of network ports to identify **accessible services** — each open port is a potential **attack surface**.



Common Service Ports

80/443	HTTP/HTTPS web services
22	SSH secure remote access
25	SMTP email transmission
53	DNS name resolution
3389	RDP remote desktop
1433/3306	SQL Server / MySQL

Defensive Application

- Close Unused Ports
Reduce **attack surface**
- Validate Firewall Rules
Verify **security policies**
- Disable Unnecessary Services
Enforce **least privilege**

❏ **Key Takeaway:** Exposed services increase attack surface — scan regularly to verify only **necessary ports** are open.

3. Sniffing & Packet Capture

Sniffers capture packets during network transmission — can be **hardware modules**, **software**, or **combined solutions**.

📄 ⚠️ **Risk:** Unencrypted sensitive data (**credentials**, **credit cards**, **passwords**) transmitted from client to server can be **intercepted and exploited**.

Popular Sniffing Tools



Tcpdump

Command-line packet analyzer



Wireshark

GUI packet analyzer (Figure 2.6)



Pcap / WinPcap / Libpcap

Packet capture libraries



IP Tools

Network utilities —
softpedia.com

Defense Strategies



Encrypt Traffic

TLS/SSL, SSH, VPN



Network Segmentation

VLANs, firewall rules



Switch Security

Port security, 802.1X



Wireless Security

WPA3, rogue AP detection

📄 **Key Takeaway:** Encrypt all sensitive traffic — even if intercepted, encrypted packets reveal nothing without the keys.

Wireshark 1.10.8 (v1.10.8-2-g52a5244 from master-1.10)

Filter: tcp.stream eq 1

No.	Time	Source	Destination	Protocol	Length	Info
101	5.60613800	10.10.10.100	10.10.10.1	TCP	1514	[TCP segment of a reassembled PDU]
102	5.60613900	10.10.10.100	10.10.10.1	HTTP	436	HTTP/1.1 401 Unauthorized (text/html)
103	5.60615900	10.10.10.1	10.10.10.100	TCP	54	warmspotMgmt > http [ACK] Seq=296 Ack=184
111	5.78797200	10.10.10.100	10.10.10.1	TCP	60	[TCP Dup ACK 102#1] http > warmspotMgmt
413	21.4221450	10.10.10.1	10.10.10.100	HTTP	388	GET /protected HTTP/1.1
414	21.4232770	10.10.10.100	10.10.10.1	TCP	1514	[TCP segment of a reassembled PDU]
415	21.4232780	10.10.10.100	10.10.10.1	HTTP	319	HTTP/1.1 401 Unauthorized (text/html)
416	21.4233150	10.10.10.1	10.10.10.100	TCP	54	warmspotMgmt > http [ACK] Seq=630 Ack=356
422	21.6107570	10.10.10.100	10.10.10.1	TCP	60	[TCP Dup ACK 415#1] http > warmspotMgmt

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-US,en;q=0.5\r\n
Accept-Encoding: gzip, deflate\r\n
Connection: keep-alive\r\n
Authorization: Basic am9lomjsb2dncw==\r\n
Credentials: joe:bloggs
[Full request URI: http://10.10.10.100/protected]
[HTTP request 2/2]
[Prev request in frame: 100]

0140 65 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b e..Conne ction: k
0150 65 65 70 2d 61 6c 69 76 65 0d 0a 41 75 74 68 6f eep-aliv e..Autho
0160 72 69 7a 61 74 69 6f 6e 3a 20 42 61 73 69 63 20 rization : Basic
0170 61 6d 39 6c 4f 6d 4a 73 62 32 64 6e 63 77 3d 3d am9lomjs b2dncw==
0180 0d 0a 0d 0a ...

HTTP Authorization header (http.authorizati... Packets: 542 · Displayed: 13 (2.4%) · Droppe... Profile: Default

Sử dụng Wireshark để bắt gói tin có chứa thông tin nhạy cảm

4. Keyloggers: Capturing User Input

Surveillance tools that record every keystroke – operating at the **input layer**, bypassing application-level encryption and security controls.

🔌 Hardware Keyloggers

Physical devices inserted between **keyboard and computer**. Requires physical access – operates outside the OS.

Attacker Advantages

- No software footprint
- Immune to **antivirus**
- Captures **pre-boot** & BIOS credentials

Limitations

- Requires **physical access**
- Data retrieval needed
- Visible on inspection

💻 Software Keyloggers

Installed via **malware**, social engineering, or remote exploitation. Hooks into OS keyboard input handling.

- Logs keystrokes to file
- Exfiltrates to **remote servers**
- Captures **screenshots**, clipboard, app data

Detection

- **Antivirus** & behavioral analysis
- **EDR** tools
- Application whitelisting



Mô đun Keylogger phân cứng và cài đặt trên máy tính để bàn

Common Types of Attacks

Common Types of Attacks

Understanding the attack landscape — from basic brute force to sophisticated nation-state campaigns.

- **Password Attacks**

Dictionary & **brute force** methods targeting authentication systems

- **Malware Attacks**

Viruses, worms, trojans & ransomware compromising system integrity

- **Denial of Service (DoS)**

Resource exhaustion attacks rendering services unavailable

- **Distributed DoS (DDoS)**

Coordinated **botnet attacks** amplifying disruption at scale

- **IP Spoofing**

Forging source addresses to impersonate trusted hosts

- **Eavesdropping**

Passive traffic interception capturing unencrypted data in transit

- **Man-in-the-Middle (MITM)**

Active interception & relay between two communicating parties

- **Email Bombing & Spam**

Overwhelming email systems to disrupt communication channels

- **Social Engineering**

Exploiting human psychology to bypass technical controls

- **Pharming**

DNS/hosts file manipulation redirecting users to malicious sites

- **Advanced Persistent Threat (APT)**

Sophisticated long-term campaigns by well-resourced threat actors

Password Attacks: Common Patterns

Despite security training, **weak passwords** remain among the most exploited vulnerabilities. Two dominant attack methods dominate the threat landscape.

Dictionary Attacks

Use precompiled lists of **common passwords**, **natural language words**, and **breach databases** with billions of credentials.

→ Common Targets

password123, **Welcome1**,
simple substitutions

→ Breach Integration

Tools like **Have I Been Pwned**
index billions of
compromised credentials

Brute Force Attacks

Exhaustively tries **all possible combinations** — success guaranteed given enough time and **compute power**.

Length	Complexity	Time to Crack
6 chars	Lowercase only	⚡ Seconds
8 chars	Alphanumeric	🕒 Hours–Days
12 chars	Mixed + symbols	🔒 Years+

 **Online Cracking**

- Targets **live auth systems**
- Constrained by **rate limiting**
- Slower — network dependent

 **Offline Cracking**

- Targets **stolen password hashes**
- **GPU-accelerated** — billions/second
- Orders of magnitude **faster**

📌 **Key Takeaway:** Weak passwords fail under guessing pressure.

Defending Against Password Attacks

Layered controls across policy, technology, and behavior raise the cost of attacks beyond adversary capabilities.

Password Policy

- **12+ characters** minimum – length beats complexity
- **Passphrases** over complex short passwords
- Screen against **breach databases**
- **No mandatory** periodic rotations

Technical Controls

Hashing & Salting	Use bcrypt , scrypt , or Argon2 + unique salts
Rate Limiting	Exponential backoff, CAPTCHA, account lockout
MFA	Authenticator apps, hardware tokens, biometrics



Credential Hygiene

- No **password reuse**
- Use a **password manager**
- Enable **MFA** everywhere



Monitoring

- **Impossible travel** detection
- Multiple **failed attempts**
- **Geo-anomaly** alerts



Defense-in-Depth

- Policy + technology
- User education
- No **single point** of failure

📌 **Key Takeaway:** Authentication security is layered, not single-measure.

Buffer Overflow: Why Memory Bugs Matter

When a program writes **more data than a buffer can hold**, adjacent memory is overwritten — potentially giving attackers control of program execution.

How Overflows Occur

- **Allocate Buffer**
Program reserves fixed memory for input
- **Validation Fails**
Input length not checked against buffer size
- **Memory Corrupted**
Excess data overwrites adjacent memory regions

❏ **Common culprits:** `strcpy`, `gets`, `sprintf` — functions with no bounds checking

Simplified Memory Layout

Region	Contents	Overflow Risk
Code	Executable instructions	Typically read-only; protected
Data	Initialized global variables	Overwrite can corrupt state
Heap	Dynamic allocations (grows ↑)	Heap overflow → pointer corruption
Stack	Function calls, locals (grows ↓)	Stack overflow → hijack return address

❏ **Key Takeaway:** Unbounded input can corrupt control flow — validate all input lengths and use memory-safe functions.

Memory Regions at Risk



Stack Overflow

- Overwrites **local variables** & **return addresses**
- Redirects **program execution flow**



Heap Overflow

- Corrupts **dynamically allocated** memory
- Overwrites **object pointers** & metadata

Buffer Overflow Defenses: Modern System Protections

Multiple overlapping protections make exploitation significantly harder. No single mitigation is perfect — but their **combination creates substantial barriers**.



DEP / NX Bit

- Memory is **writable OR executable** — never both
- Enforced at **hardware level** (CPU NX bit)
- Injected code **cannot execute** from stack/heap



ASLR

- Randomizes **stack, heap, code** locations at runtime
- Attackers can't predict **exact memory addresses**
- Requires **PIE** (position-independent executables)



Stack Canaries

- Special value placed **between buffer & return address**
- Verified before function returns
- Overflow detected → program **terminates safely**



Compiler Hardening

- Stack protector** flags enabled at compile time
- Control flow integrity** checks
- Warnings for **dangerous functions** (strcpy, gets)



Safe Libraries & Languages

- Rust, Go, Java, C#** — automatic bounds checking
- In C/C++: use **strncpy, snprintf** with length limits
- Validate **all inputs** before processing

Protection	What It Prevents	Limitation
DEP/NX	Executing injected shellcode	Bypassed by ROP chains
ASLR	Reliable address prediction	Weakened without full PIE
Stack Canaries	Silent return address overwrite	Bypassed if canary is leaked
Safe Languages	Bounds violations at source level	C/C++ legacy code still at risk



Key Takeaway: Layer your defenses — exploitability drops dramatically when DEP + ASLR + canaries are all active. Legacy systems missing any layer face **elevated risk**.

Unvalidated Input: A Common Root Vulnerability

All external input is untrusted until validated. Unvalidated input enables injection attacks, buffer overflows, logic bypasses, and more.

Input Sources & Risks

-  **Web Forms & APIs**
Headers, cookies, user-supplied data
-  **URLs & Query Strings**
GET parameters, path variables
-  **File Uploads**
Names, content, metadata
-  **Network Protocols**
Packets, serialized objects
-  **CLI Arguments**
Parameters passed to programs
-  **Database & Env Vars**
External queries, config values

Validation Checklist

 Length Enforce min/max limits	 Type Integer, string, date
 Format Regex / parser rules	 Allowlist Permit known-good only
 Encoding Handle special chars safely	

☐ **Key Takeaway:** Validate **early**, validate **consistently** — use **allowlists** over blocklists.

SQL Injection: When Data Becomes Code

Demonstrating **unvalidated input exploitation** through **SQL code injection** to steal or destroy database data.

The Vulnerable Query

SELECT * FROM tbl_products WHERE product_name LIKE '%' + keyword + '%'		
tbl_products Product information table	product_name Product name field	keyword User-provided search term — unvalidated

Normal vs Malicious Usage

Scenario	User Input	Resulting Query	SQL Executed	Result
✔ Normal Usage	iPhone 7	Standard search	SELECT * FROM tbl_products WHERE product_name LIKE '%iPhone 7%'	Returns matching records or empty set
☠ Malicious Injection	iPhone X';DELETE FROM tbl_products;--	Injected batch commands	SELECT * FROM tbl_products WHERE product_name LIKE '%iPhone X'; DELETE FROM tbl_products;--'	ALL RECORDS DELETED

📄

Attack Mechanism:

- **Semicolon (;)** enables batch SQL execution — multiple statements run at once
- **DELETE FROM tbl_products** removes all data from the table instantly
- **Double dash (--)** comments out remaining SQL, hiding the attack and eliminating syntax errors
- Can also inject **INSERT / UPDATE** commands to add or modify data in any table

📄

⚠ Critical Vulnerability: The application concatenates **untrusted user input** directly into SQL queries without **validation** or **parameterization**.

SQL Injection: What Attackers Can Achieve

Successful exploitation can compromise **entire databases**, expose sensitive data, and even enable **OS-level command execution**.



Auth Bypass

Login without valid credentials — gain **admin access**



Data Theft

UNION/blind injection to extract any accessible table



Data Modification

Alter **prices, balances, privileges** via INSERT/UPDATE



Data Destruction

DROP tables or TRUNCATE critical data — **severe disruption**



OS Command Exec

xp_cmdshell escalates DB compromise → full server control

Privilege Amplification

✗ Excessive Privileges

- Admin/DBA database account
- Access to all tables
- Write + execute permissions
- OS procedure access enabled

✓ Least Privilege

- Dedicated app-only account
- Restricted to required tables
- Read-only where possible
- Dangerous features disabled

📌 **Key Takeaway:** SQLi impact is **dramatically amplified** by excessive database privileges — apply **least privilege** to limit blast radius.

Preventing SQL Injection

SQL injection is **entirely preventable** through proper coding practices and defense-in-depth controls. Implement **multiple complementary layers** rather than relying on any single technique.

Primary Defense: Parameterized Queries

How It Works

Separates **SQL structure** from **user data** using placeholders – input is **never interpreted as code**

Why It's Best

Works across **all platforms** and languages – eliminates injection **at the source**

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM users WHERE username = ?");
stmt.setString(1, userInput);
ResultSet rs = stmt.executeQuery();
```

Supporting Controls

- **Stored Procedures**
Safe only when using **parameters** – not dynamic SQL concatenation
- **Input Validation**
Validate **format, type, length** – defense-in-depth, not sole protection
- **Least Privilege**
Restrict DB accounts to **minimum permissions** – no admin rights
- **Error Handling**
Show **generic errors** to users – log details **securely** server-side

❏ **Key Takeaway: Parameterized queries + least privilege** = strong baseline defense against SQL injection.

Denial of Service (DoS)

DoS attacks disrupt **availability** by exhausting finite resources or crashing services — not stealing data.

Resource Exhaustion Targets



Bandwidth



Memory



App Threads



CPU Cycles



Connections



Disk I/O

Attack Classifications

⚡ Logic-Based

- Exploits **bugs or protocol weaknesses**
- Small input → disproportionate damage
- Examples: **malformed packets**, algorithmic complexity, expensive DB queries

🌊 Flooding-Based

- Overwhelms with **sheer traffic volume**
- Requires attacker resources or **amplification**
- Examples: **SYN flood**, ICMP flood, HTTP flood, UDP amplification

📌 **Key Takeaway:** Availability fails when resources are **exhausted** or services **crash**.

SYN Flood: Exploiting TCP Handshake State

SYN floods exploit TCP's stateful handshake by filling the server's **connection backlog** with **half-open connections**, blocking legitimate users.

Normal TCP Handshake

01

Client → SYN

Initiates connection request

02

Server → SYN-ACK

Allocates state, awaits confirmation

03

Client → ACK

Connection **established**

SYN Flood Attack

- Flood with SYNs
High-volume SYN packets, often **spoofed IPs**
- Never send ACK
Connections stay **half-open**
- Backlog exhausted
Legitimate users **refused**

Defenses



SYN Cookies

Encode state in **sequence number** — no storage until ACK arrives



Tuning Parameters

Larger **backlog**, shorter **timeouts**



Filtering & Proxies

Rate-limit SYNs, proxy handshakes, block spoofed IPs

📌 **Key Takeaway:** Attacking protocol **state** can deny access without breaking the server.

Smurf Attack: ICMP Broadcast Amplification

Attackers exploit **IP broadcast addressing** and **ICMP echo behavior** to turn entire networks into unwitting amplifiers — flooding victims with traffic far exceeding the attacker's bandwidth.

Attack Mechanics

1. Send **ICMP echo request** to broadcast address
2. Spoof source IP → victim's address
3. All hosts receive and reply to **victim**
4. Amplification = number of responding hosts

100 hosts

= **100:1** amplification factor

Multiple networks

= **even greater** amplification

Defense Strategies



Disable Directed Broadcasts

Drop packets to broadcast addresses from external networks.

`no ip directed-broadcast`



Filter Spoofed Packets

Ingress/egress filtering drops packets with illegitimate source IPs. See **RFC 2827 / BCP 38**.



Rate-Limit ICMP

Aggressively limit **ICMP echo** traffic at network edges.

 **Key Takeaway:** Amplification attacks multiply attacker traffic into victim traffic — **1 packet in, 100+ packets out**.

Distributed DoS (DDoS): Scale and Distribution

Multiple compromised systems simultaneously target a single victim — coordination transforms attack dynamics fundamentally, leveraging **botnets** of thousands to millions of devices.

DDoS vs. Single-Source DoS



Massive Scale

Attacks exceeding **1 Tbps** —
overwhelming even major
providers



Unblockable

Thousands of source IPs —
blocking causes collateral
damage



Geographic Spread

Nodes across **multiple countries
& ISPs** — no regional filter works



Multi-Vector

Network + transport + application
layer attacks combined

Defense Strategies

Anycast & CDN

Distribute traffic across **global
infrastructure** — no single target

Scrubbing Centers

Route traffic through **high-
capacity filters** — clean traffic
forwarded

ISP Cooperation

Filter **upstream at source** —
requires multi-party
coordination

Overprovisioning

Excess capacity absorbs small
attacks — impractical at large
scale

❏ **Key Takeaway:** DDoS is a scale problem requiring **layered, distributed response** — no single mitigation suffices.

Botnets: The Engine Behind DDoS Attacks

Networks of **compromised devices** (bots/zombies) controlled by a **botmaster** — ranging from hundreds to **millions of hosts** including PCs, servers, IoT, and mobile devices.

Architecture Models

Model	Resilience	Control
Centralized (HTTP/IRC)	Low — single point of failure	Real-time, efficient
Peer-to-Peer	High — no central server	Slow, harder to manage
Hybrid	Medium — balanced	Flexible

Botnet Applications

 DDoS Attacks Massive floods; DDoS-for-hire services	 Spam Bulk email from distributed sources
 Malware Distribution Host & spread infection chains	 Credential Stuffing Evade rate limiting at scale
 Cryptojacking Hijack CPU for mining	 Click Fraud Fake ad clicks & traffic

 **Key Takeaway:** Botnets **industrialize cyberattacks** — turning compromised devices into scalable, for-hire attack infrastructure.

Mitigating DoS/DDoS: Practical Controls

Defense-in-depth across **network**, **application**, and **organizational** layers — no single control is sufficient.

Prevention Controls

Network Edge Filtering

- **Ingress/egress filters** block spoofed packets
- Rate limit **ICMP**, **UDP amplification** vectors
- Block known **malicious sources**

Application Hardening

- **Connection limits** & request timeouts
- **Caching** for static content
- Prevent **algorithmic complexity** attacks

Architecture Resilience

- **Redundancy** & geographic distribution
- Sufficient **capacity headroom**
- **Graceful degradation** under load

Detection & Response

Anomaly Detection

- Establish **traffic baselines**
- Monitor spikes, protocol ratios, **geo changes**
- **IDS/IPS** for attack signatures

Traffic Scrubbing

- Route via **scrubbing centers** during attacks
- Filter malicious traffic at scale
- Forward only **legitimate requests**

CDN & Anycast

- Distribute load across **global infrastructure**
- Absorb attacks with **aggregate capacity**

ISP Coordination

- Pre-establish **upstream relationships**
- Filter closer to **attack sources**
- Prevent bandwidth exhaustion

 **Key Takeaway:** Mitigation requires **prevention + detection + upstream support**.

IP Spoofing: Forging Source Identity

Packets with **forged source IPs** — bypassing authentication, enabling amplification attacks, and obscuring attacker identity. Works because **IP has no source validation**.

How It Works

- **Forge Source IP**
Attacker sets arbitrary source address in packet header
- **Router Forwards It**
Routers route by **destination only** — no source verification
- **Victim Responds**
Replies go to **spoofed victim IP**, not the attacker

Common Attack Uses

Amplification

DNS, NTP, Smurf attacks

SYN Floods

Hide attacker location

Bypass ACLs

Circumvent IP-based controls

Anti-Spoofing Defenses

BCP 38 Ingress Filtering

Drop packets whose **source IP couldn't originate** from that network (e.g., block internal IPs arriving from internet)

Egress Filtering

Ensure **only legitimate org IPs** leave the network — prevents compromised hosts from spoofing

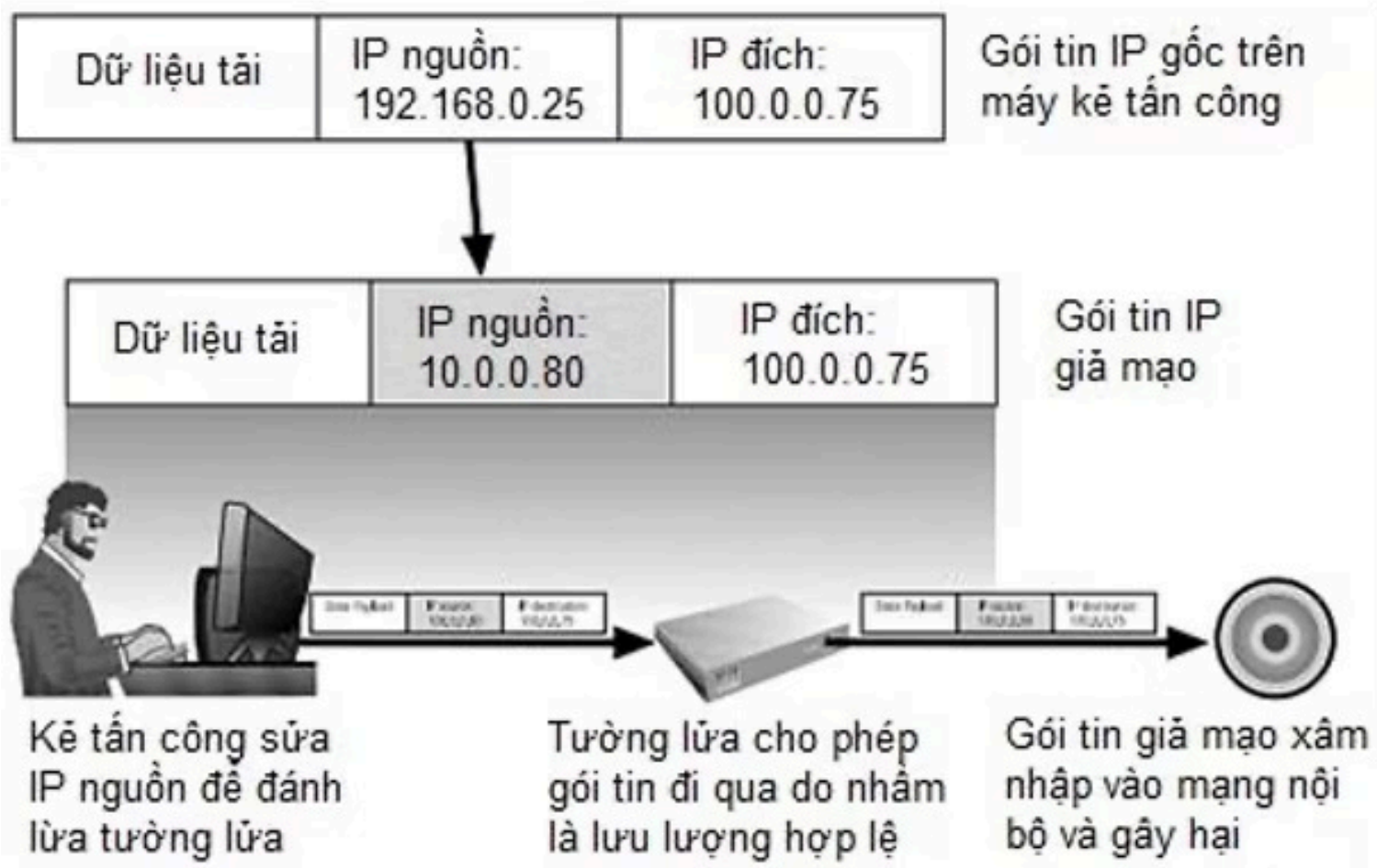
uRPF (Unicast Reverse Path Forwarding)

Router verifies incoming packets have a **valid return route** from claimed source

Cryptographic Authentication

Never trust source IP alone — use TLS, certificates, or signed tokens for identity

❏ **Key Takeaway: Trust based on IP alone is fragile** — network-layer identity requires cryptographic verification.



Minh họa tấn công giả mạo địa chỉ IP

Eavesdropping: Listening to Communications

Passively capturing network traffic without authorization. **Encrypted traffic** becomes unreadable noise; **plaintext protocols** expose everything.

Wired Networks

- Physical network **taps** on cables
- **ARP spoofing** on switched networks
- Abusing **SPAN/mirror ports**
- Compromising **routers & firewalls**

Wireless Networks

- **Open Wi-Fi** – trivial capture, no access needed
- **Weak WPA2/WPA3** – offline password cracking
- **Public Wi-Fi** – always treat as hostile
- Insider threat – legit users capturing others' traffic



Encryption

- **TLS/HTTPS** for web
- **SSH** for remote access
- **VPN** for network comms



Wireless Security

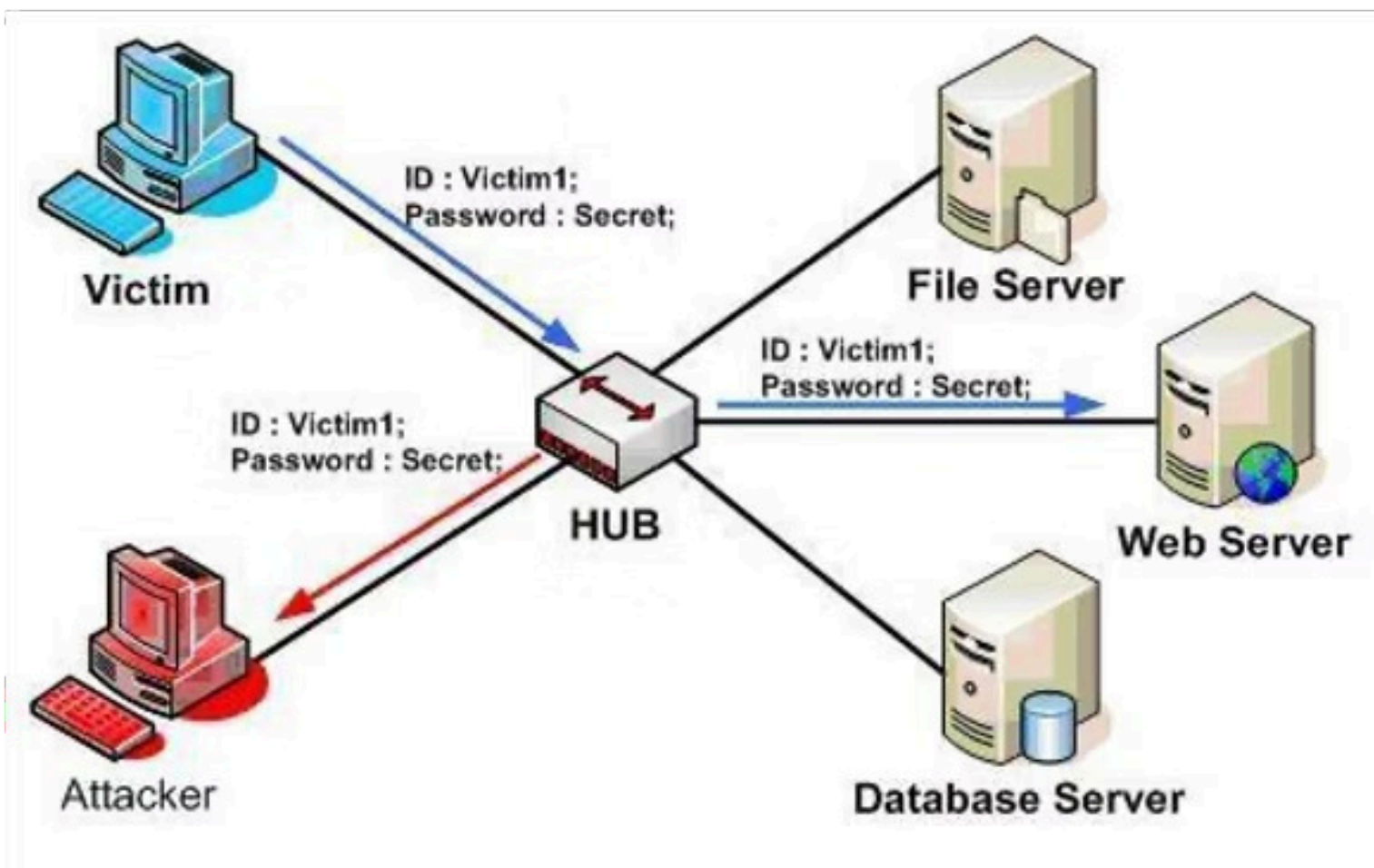
- **WPA3** preferred
- **802.1X** for enterprise
- Disable **WPS**



Segmentation

- Isolate **sensitive systems**
- Limit traffic **visibility**
- Controlled **access points**

 **Key Takeaway:** Encryption turns interception into noise.



Một mô hình tấn công nghe lén

Man-in-the-Middle (MITM): Intercepting and Relaying Messages

An attacker positions between two parties — **intercepting, reading, and modifying messages** while victims believe they're communicating directly.



Mô hình chung của tấn công kiểu người đứng giữa

Attack Scenarios

- | | | | |
|---|---|---|---|
|  Rogue Wi-Fi AP
Fake network (e.g., "StarbucksWiFi") routes all traffic through attacker |  ARP Spoofing
Poisons local subnet to intercept traffic between hosts and gateway |  DNS Manipulation
Redirects legitimate domains to attacker-controlled servers |  TLS/SSL Downgrade
Forces weak or no encryption during connection setup |
|---|---|---|---|

☐ **Key Takeaway:** MITM succeeds when identity is not strongly verified.

Preventing MITM: Verify Identity + Protect Integrity

Cryptographic Controls

TLS 1.2 / 1.3

Validate certificates against **trusted CAs** · Verify domain name match

Certificate Pinning

Reject connections not matching **expected cert/public key** · Blocks fraudulent CA issuance

HSTS

Force **HTTPS-only** connections · Prevents **downgrade attacks** · Preload lists for first visit

Public Key Infrastructure (PKI)

Trust Chain

Root CAs → Intermediate CAs → End-entity certs

Certificate Management

Keep **CA stores updated** · Revoke compromised certs · Monitor **transparency logs**



Mutual TLS (mTLS)

Both client & server present **certificates**

- Prevents server impersonation
- Blocks unauthorized clients
- Required for **APIs & enterprise**



Digital Signatures

Sign critical messages for **integrity & authenticity**

- Prevents modification
- **Non-repudiation**
- Used in S/MIME, PGP, code signing



VPN Protection

Encrypts **all traffic** between endpoints

- Guards against **local network MITM**
- Required on public **Wi-Fi**
- Mandatory for remote access

❏ **Key Takeaway: Encryption + authentication** = silent interception becomes impossible.

Social Engineering: Attacking the Human Layer

Attackers exploit **human psychology** — not software. Trust, authority, urgency, and fear bypass even the strongest technical defenses.

Psychological Triggers

- **Authority**
Impersonating executives, IT, or law enforcement
- **Urgency**
"Account locked in 30 minutes — act now!"
- **Fear**
Threats of suspension, legal action, or job loss
- **Curiosity**
"See who viewed your profile!" — sensational lures
- **Reward**
Prizes, money, or exclusive access — too good to be true
- **Trust**
Rapport-building, fake familiarity, impersonating colleagues

Common Attack Vectors

- **Phishing**
Fake emails mimicking real orgs — steal credentials, link to fake login pages. **Spear phishing** targets individuals.
- **Vishing**
Phone calls posing as **IT support, banks, or executives** — request passwords, codes, or malware installs.
- **Pretexting**
Fabricated scenarios to extract info — e.g., fake employee calling HR for credential reset.
- **Baiting**
Enticing offers (free software, USB drives) that **deliver malware** when accessed.

Defense Strategy

- **Security Training**
Regular awareness programs + **simulated phishing** campaigns
- **Verification Protocols**
Always verify payment/access changes via **known channels**
- **Technical Controls**
Email filtering, caller ID checks, **MFA**
- **Reporting Culture**
Blame-free incident reporting — catch attacks **early**

 **Key Takeaway:** People are a core part of the security perimeter — train them like a technical control.

Nigeria 419 Scam: A Classic Social Engineering Pattern

Named after Nigeria's criminal code section — exploits **greed**, **optimism bias**, and **desire to help**. The pattern applies to many modern attacks.

The Classic Pattern

01

Initial Contact

Unsolicited message from a "prince," official, or business person

02

The Offer

Help transfer large funds → receive a **large % cut**

03

Upfront Fee

Small payment required to "unlock" the larger reward

04

Escalation

Sunk cost keeps victims paying to recoup losses

Why It Works

Optimism Bias

People **overestimate luck** and underestimate fraud risk

Urgency

Time-limited offers prevent careful evaluation

Sunk Cost

Prior payments create **psychological investment**

Embarrassment

Victims **don't report** — letting scammers continue

Defense Strategy

Skepticism of unsolicited offers · Verify independently · No legitimate deal requires upfront fees · "Too good to be true" usually is · Encourage judgment-free reporting

Modern Variations

- Lottery / Prize
Fees required to claim "winnings"
- Romance Scams
Relationship built, then money requested
- Work-From-Home
Easy income promises
- Fake Investment
"Guaranteed" high returns
- Inheritance
Unknown relative left you money

Pharming: Redirecting Users to Fake Sites

Unlike **phishing**, pharming uses **technical manipulation** of name resolution to silently redirect users to attacker-controlled sites — no deception required.

Endpoint-Based Attacks

Hosts File Modification

Malware maps **legitimate domains** → **malicious IPs**, bypassing DNS entirely

DNS Settings Tampering

Changes system DNS to **attacker-controlled servers** returning fake IPs

Browser Modification

Malicious **extensions** redirect URLs or proxy traffic through attacker systems

DNS Infrastructure Attacks

Cache Poisoning

Injects **false DNS records** into caching servers — persists until cache expires

Server Compromise

Direct control of **authoritative/recursive DNS** — affects entire orgs or ISPs

Defense Strategies



Endpoint Security

Antivirus, anti-malware, restricted admin access, **system file monitoring**



TLS Certificate Validation

HTTPS fails if DNS is poisoned — attackers can't obtain valid certs. **Heed browser warnings.**



DNSSEC & Trusted Resolvers

Cryptographic authentication of DNS responses + DNS query **anomaly monitoring**

 **Key Takeaway:** Pharming attacks **trust in name resolution** — defenses must span endpoint, DNS infrastructure, and TLS layers.

APT: Advanced Persistent Threat

Sophisticated, multi-stage campaigns by **nation-state actors** or **organized cybercrime groups** targeting specific high-value organizations over extended periods.



Advanced

- Custom malware & **zero-day exploits**
- Living-off-the-land techniques
- Tailored social engineering
- Adaptive evasion tactics



Persistent

- **Dwell time:** months to years
- Multiple redundant backdoors
- Stealth over speed
- Re-infection capabilities



Targeted

- **IP theft** & espionage
- Competitive intelligence
- Critical infrastructure
- Strategic, not opportunistic

Defense Requirements

Detect

- **SIEM** & UEBA analytics
- Threat hunting
- **EDR** on endpoints

Contain

- **Network segmentation**
- Least privilege access
- Zero trust architecture

Respond

- Robust **IR capabilities**
- Threat intelligence feeds
- **Assume compromise** mindset

 **Key Takeaway:** APTs are long-term **campaigns**, not single events — detect and respond before objectives are achieved.

Malware: Malicious Software Overview

Any program designed to **harm systems**, **steal data**, or **gain unauthorized access**. Modern malware combines multiple behaviors, creating hybrid threats.

Malware Objectives



Data Theft
Credentials, IP, personal data



Disruption
Corrupt files, disable systems



Surveillance
Keylogging, audio/video capture



Monetization
Ransomware, crypto mining



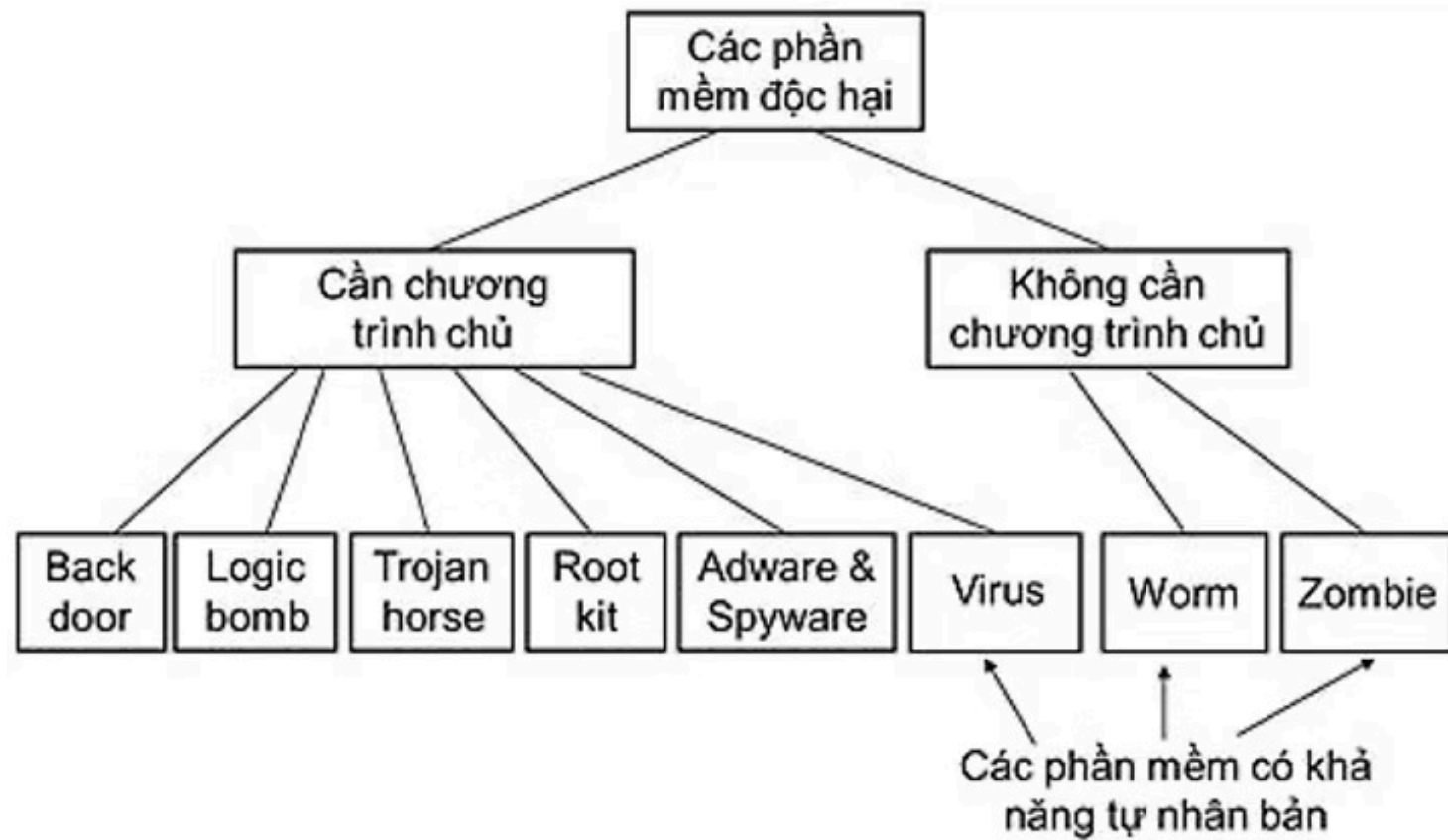
Remote Control
Backdoors, botnets

Classification Framework

Type	Spreads	Examples
Virus	User action	File, boot sector, macro
Worm	Autonomous	Network exploitation
Trojan	Disguised	Fake apps, installers
Rootkit	Post-exploit	Kernel-level hiding
Ransomware	Phishing/worm	File encryption, extortion
Spyware	Bundled/drive-by	Keyloggers, stealers

📌 ⚠️ Real-world malware is **hybrid** — worms deliver trojans that install rootkits and backdoors simultaneously.

📌 **Key Takeaway:** Malware categories overlap in real-world campaigns.



Các dạng phần mềm độc hại

Virus: Infecting Other Programs

Malicious code that **replicates by attaching to legitimate programs or files** — requires host execution to spread.

Infection Lifecycle

- **Execute**
User runs **infected program** or opens **infected document**
- **Activate**
Virus code runs, performs **malicious payload**
- **Propagate**
Searches for **uninfected targets** — files, programs, boot sectors
- **Replicate**
Copies itself to targets, creating **new infections**
- **Conceal**
Returns control to **host program** to avoid detection

Defense Strategies



Antivirus

Signature + heuristic detection, real-time scanning



Macro Restrictions

Disable by default, enable only for **trusted sources**



Patch Management

Update OS & apps — closes **known vulnerabilities**



Safe Handling

Scan before opening, avoid **untrusted executables**

Virus Types



Boot Sector

Infects **master boot record** — executes before OS loads. Largely obsolete with **UEFI Secure Boot**.



File Virus

Attaches to **.exe, .com, .dll** files. Spreads via downloads, **USB drives**, shared files.



Macro Virus

Embeds in **Office/PDF macros**. Triggered on document open. Declined as **macros disabled by default**.



Email Virus

Spreads via **infected attachments**. Uses **social engineering** + address books to propagate.

Worm: Self-Propagation Across Networks

Self-contained malicious programs that spread **automatically across networks** — no host file or user interaction required. Can infect **thousands of systems within hours**.

Propagation Mechanisms

**Vulnerability Exploitation**
Scans for unpatched systems — **buffer overflows**, insecure configs, exposed services

**Credential Spreading**
Stolen credentials, default passwords, or brute-force lists to authenticate laterally

**Email & Messaging**
Auto-sends to **contact lists**; distribution is automated even if opening requires user action

**File Sharing**
Copies to **shared folders**, removable media, and P2P networks

**Rapid Patching**
Close exploit windows immediately

**Segmentation**
Limit lateral movement paths

**IDS/IPS**
Block exploitation attempts

**Egress Filtering**
Prevent outbound scanning

**Least Privilege**
Minimize post-compromise spread

 **Key Takeaway:** Worms scale by **automation** and **network reach** — defense requires equal speed.

#	Malware	Year	Type	Estimated cost	Reach / scale	Behavior	Notable impacts
1	Mydoom (Novarg)	2004	Worm (mass-emailing)	\$38B (inflation-adjusted \$52.2B)	At peak: 25% of all emails ; still ~1% of phishing emails	Scrapes email addresses → mass-mails copies; forms botnet for DDoS	Developer never caught (reward offered)
2	Sobig (A–F, esp. Sobig.F)	2003	Worm	\$30B	Global impact (Canada, UK, US, Europe, Asia)	Email attachments disguised as legitimate software	Disrupted Air Canada ticketing; creator never caught
3	Klez	2001	Worm	\$19.8B	~7.2% of all computers (~7 million PCs)	Fake/spoofed emails; infects files; spreads within networks	Many variants; persisted for years
4	ILOVEYOU (Loveletter)	2000	Email-spreading virus/worm	\$15B	10+ million PCs shortly after May 4 release	“Love letter” attachment → mails itself to all contacts	Created by Onel de Guzman (Philippines); aimed at stealing passwords
5	WannaCry	2017	Ransomware	\$4B	150 countries, ~200,000 computers	Encrypts files and holds them hostage; hit outdated systems hard	Stopped after a UK security researcher found a “kill switch” method
6	Zeus	2007	Banking malware / botnet-based theft tool	\$3B (inflation-adjusted \$3.7B)	In 196 countries; 76,000 computers ; ~2,500 orgs; claimed 88% of Fortune 500 affected	Botnet controlled by a “bot master” to steal/transfer money	Many arrests (2010); source code influenced later botnets
7	Code Red	2001	Worm	\$2.4B	975,000 hosts	Memory-resident; defaces web pages; used infected hosts for DDoS	DDoS targeted whitehouse.gov ; White House changed IP to respond
8	SQL Slammer (Sapphire)	2003	Worm	Listed \$1.2B (also cited \$750M across 200k users)	~200,000 users affected (as stated)	Random IP scanning; exploits vulnerabilities; causes massive traffic	Took ATMs offline ; reappeared in 2016 from multiple countries’ IPs
9	CryptoLocker	2013	Ransomware	\$665M (estimate)	Up to 250,000 machines ; (estimate used: ~5,000 companies)	Encrypts files; distributed using Gameover Zeus botnet	Ransom note + payment window; cost estimate derived from avg ransomware loss
10	Sasser	2004	Worm	\$500M (some reports claimed \$18B)	“Crashed millions of PCs”	Self-spreading worm (details not expanded in the text)	Written by Sven Jaschan (Germany); arrested after \$250k bounty

Trojan, Backdoor, Rootkit: Key Differences

Three malware types focused on **hidden access** and **evasion** — not propagation.

Characteristic	Trojan	Backdoor	Rootkit
Purpose	Disguise malicious code as legitimate software	Provide hidden remote access	Hide malware & maintain privileged access
Entry Method	User installs voluntarily	Installed by other malware or exploits	Requires privileged access
Stealth Level	Social engineering for installation	Hidden from users, but detectable	Deep system-level hiding
Detection	Antivirus , behavioral analysis	Network monitoring , process inspection	Rootkit scanners , memory forensics

Modern Malware Families by Motivation

Contemporary malware is increasingly **financially motivated** — each family optimized for a specific **monetization strategy**.



Bots

Join **botnets** for **DDoS**, spam, **credential stuffing**, click fraud, crypto mining



Spyware

Exfiltrates **credentials**, banking details, **keystrokes** — sold on dark markets



Adware

Hijacks **search results**, tracks browsing, installs without consent

Ransomware: Double Extortion

01

Initial Access

Phishing, exploits, or **credential theft**

02

Lateral Movement

Reconnaissance to maximize impact

03

Data Exfiltration

Copy data for **extortion leverage**

04

Disable Defenses

Neutralize **backups** & security tools

05

Encrypt & Extort

Deploy **ransomware**, demand payment, threaten **data leak**

Defending Against Malware: Defense in Depth

No single control provides complete protection. Layered, overlapping defenses ensure resilience when individual controls are bypassed.

Prevention Controls

→ Reduce Attack Surface

Disable unused services · close ports · restrict admin access

→ Patch Management

Automated updates · prioritize **actively exploited CVEs**

→ Endpoint Protection (EDR)

Behavioral analysis · zero-day detection · automated response

→ Application Control

Allowlisting blocks unauthorized software execution

Detection & Response

→ SIEM Monitoring

Aggregate & correlate events · detect **indicators of compromise**

→ Network Segmentation

Isolate critical systems · slow **lateral movement**

→ DNS Filtering

Block **malicious domains** · disrupt C2 communications

→ Backup & Recovery

Offline, immutable backups · tested regularly · IR plans

Security is a continuous process, not a one-time setup. Threats evolve constantly — defenses must too.

📌 **Key Takeaway:** Prevention + detection + recovery = resilient systems.

Review Questions

1. What is a system weakness? List the causes of the existence of weaknesses in a system.
2. What is a security vulnerability? In which system component do vulnerabilities most commonly exist?
3. State common types of security vulnerabilities found in operating systems and application software.
4. What is a threat? Explain the relationship between vulnerabilities and threats.
5. What is an attack? How can the likelihood of being attacked be reduced?
6. Describe the four main categories of attacks and the two types of attacks: active and passive.
7. State the objectives and types of password attacks.
8. What is SQL injection? State the causes of SQL injection vulnerabilities. What actions can SQL injection enable an attacker to perform on the victim system?
9. State the countermeasures for preventing SQL injection attacks.
10. Draw a diagram and describe the mechanism of a SYN Flood attack and its countermeasures.
11. Draw a diagram and describe the mechanism of a Smurf attack and its countermeasures.
12. Draw a diagram and describe the scenarios of direct DDoS attacks and indirect (reflective) DDoS attacks.
13. Describe the mechanism and countermeasures of a Man-in-the-Middle attack.
14. Who/what are the targets of social engineering attacks? Describe the scenario of the Nigeria 4-1-9 scam.
15. What is a pharming attack? Describe the types/forms of pharming attacks.
16. What is an APT (Advanced Persistent Threat) attack? Describe the key characteristics of APT attacks.
17. What is malware? Classify the types of malware.
18. What is a computer virus? State infection methods and types of viruses.
19. What is a Trojan (Trojan horse)? Describe how a Trojan operates.
20. What is a computer worm? State the fundamental difference between worms and viruses. List methods by which worms spread.
21. What is a Zombie/Bot? Describe how a Zombie/Bot operates.
22. State the general principles for malware prevention. Why should only one real-time malware scanning/antivirus program be installed and running at a time?